

Team Note of cpp = competitive python programming

competitive(jwpassion1), python(yyyy7089), programming(mathdong01)

Compiled on October 1, 2026

Contents

1 Python

1.1	Segment Tree	1
1.2	Lazy Segment Tree	2
1.3	Kth Number PST	3
1.4	O(NlogN) LIS	4
1.5	LCS	4
1.6	Disjoint Set	5
1.7	Bellman Ford	5
1.8	ETT & HLD	6
1.9	SCC	7
1.10	2-SAT	8
1.11	Miller Rabin	8
1.12	Pollard Rho	9
1.13	Euler Phi	9
1.14	Euler Phi (up to N)	9
1.15	KMP	9
1.16	Manacher	10
1.17	CCW & Convex Hull	10
1.18	Rotating Calipers	11
1.19	Polygon Overlap Area	11
1.20	Ray Casting	14
1.21	Min Enclosing Circle	17
1.22	Dinic	18
1.23	Tutte Matrix	19

2 Data Structure

2.1	PBDS	19
-----	------	----

3 Graph & Flow 19

3.1	이분 매칭 (Kuh, 느림)	19
-----	-----------------	----

4 Fast Fourier Transform 20

4.1	Fast Fourier Transform	20
-----	------------------------	----

5 ETC 20

5.1	Template	20
5.2	Useful Stuff	21
5.3	자주 쓰이는 문제 접근법	21
5.4	DP 최적화 접근	22

1 Python

1.1 Segment Tree

```
class segtree:
    def __init__(self, N):
        self.size = 1 << math.ceil(math.log2(N))
        self.tree = [0] * (self.size << 1)

    def set(self, idx, val):
        self.tree[idx | self.size] = val
        return

    def init(self):
        for i in range(self.size-1, 0, -1):
            self.tree[i] = self.tree[i << 1] + self.tree[i << 1 | 1]
        return
```

```

def query(self, left, right):
    left |= self.size
    right |= self.size
    ret = 0
    while left ≤ right:
        if left & 1:
            ret += self.tree[left]
            left += 1
        if ~right & 1:
            ret += self.tree[right]
            right -= 1
        left >>= 1
        right >>= 1
    return ret

def update(self, idx, val):
    idx |= self.size
    self.tree[idx] = val
    while idx > 1:
        self.tree[idx >> 1] = self.tree[idx] + self.tree[idx ^ 1]
        idx >>= 1
    return

```

INF = 1 << 31

1.2 Lazy Segment Tree

```

class lazyseg:
    def __init__(self, N):
        self.h = math.ceil(math.log2(N))
        self.size = 1 << self.h
        self.segtree = [0 for _ in range(self.size << 1)]
        self.length = [1 for _ in range(self.size << 1)]
        self.lazy = [0 for _ in range(self.size)]

    def assign(self, i, val):
        self.segtree[i | self.size] = val

    def init(self):

```

```

        for i in reversed(range(1, self.size)):
            self.segtree[i] = self.segtree[i << 1] +
            self.segtree[i << 1 | 1]
            self.length[i] = self.length[i << 1] + self.length[i
            << 1 | 1]

    def propagate(self, i, val): # node update
        self.segtree[i] += val * self.length[i]
        if i < self.size:
            self.lazy[i] += val

    def push(self, i):
        self.propagate(i << 1, self.lazy[i])
        self.propagate(i << 1 | 1, self.lazy[i])
        self.lazy[i] = 0

    def pull(self, i): # node + node -> node
        self.segtree[i] = self.segtree[i << 1] + self.segtree[i <<
        1 | 1]

    def update(self, left, right, val):
        left |= self.size
        right |= self.size
        for i in reversed(range(1, self.h + 1)):
            if not left >> i << i = left:
                self.push(left >> i)
            if not right + 1 >> i << i = right + 1:
                self.push(right >> i)
        l, r = left, right
        while l ≤ r:
            if l & 1:
                self.propagate(l, val)
                l += 1
            if ~r & 1:
                self.propagate(r, val)
                r -= 1
            l >>= 1
            r >>= 1
        for i in range(1, self.h + 1):
            if not left >> i << i = left:
                self.pull(left >> i)

```

```

        if not right + 1 >> i << i == right + 1:
            self.pull(right >> i)

    def query_point(self, i):
        i |= self.size
        for j in reversed(range(1, self.h + 1)):
            self.push(i >> j)
        return self.segtree[i]

    def query(self, left, right):
        l, r = 0, 0
        left |= self.size
        right |= self.size
        for i in reversed(range(1, self.h + 1)):
            if not left >> i << i == left:
                self.push(left >> i)
            if not right + 1 >> i << i == right + 1:
                self.push(right >> i)
        while left <= right:
            if left & 1:
                l += self.segtree[left]
                left += 1
            if ~right & 1:
                r += self.segtree[right]
                right -= 1
            left >>= 1
            right >>= 1
        return l + r

```

1.3 Kth Number PST

```

n = int(input())
l = list(map(int, input().split()))
lg = 1 << math.ceil(math.log2(n))
slg = math.ceil(math.log2(n))

segtree = []
values = []
for i in l:
    values.append(i)
    segtree.append((-1, -1))

```

```

for i in range(lg - len(l)):
    values.append(0)
    segtree.append((-1, -1))

for i in range(lg-1):
    segtree.append((i*2, i*2+1))
    values.append(values[i*2] + values[i*2+1])

roots = [lg*2-2]

def query(L, R, x):
    Li = roots[x]; Ri = roots[x]
    depth = 1
    tot = 0
    while depth <= slg:
        if L >> (slg - depth) == R >> (slg - depth):
            if (L >> (slg - depth)) & 1:
                Li = segtree[Li][1]
                Ri = segtree[Ri][1]
            else:
                Li = segtree[Li][0]
                Ri = segtree[Ri][0]
        else:
            if (L >> (slg - depth)) & 1:
                Li = segtree[Li][1]
            else:
                if (L >> (slg - depth)) + 1 != (R >> (slg - depth)):
                    tot += values[segtree[Li][1]]
                    Li = segtree[Li][0]
                if (R >> (slg - depth)) & 1:
                    if (L >> (slg - depth)) + 1 != (R >> (slg - depth)):
                        tot += values[segtree[Ri][0]]
                        Ri = segtree[Ri][1]
                    else:
                        Ri = segtree[Ri][0]
            depth += 1
    if Li == Ri:
        return tot + values[Li]
    else:

```

```

        return tot + values[Ri] + values[Li]

def update(i, x):
    q = []
    idx = roots[-1]
    depth = 1
    while depth ≤ slg:
        if (i >> (slg - depth)) & 1:
            q.append((0, segtree[idx][0]))
            idx = segtree[idx][1]
        else:
            q.append((1, segtree[idx][1]))
            idx = segtree[idx][0]
        depth += 1

    values.append(x)
    segtree.append((-1, -1))
    while len(q) > 0:
        qb, qi = q.pop()
        x += values[qi]
        values.append(x)
        if qb == 0:
            segtree.append((qi, len(segtree)-1))
        else:
            segtree.append((len(segtree)-1, qi))
    roots.append(len(segtree)-1)

for j in range(int(input())):
    i, *l = map(int, input().split())
    if i == 1:
        update(l[0]-1, l[1])
    elif i == 2:
        print(query(l[1]-1, l[2]-1, l[0]))

```

1.4 O(NlogN) LIS

```

n = int(input())
l = list(map(int, input().split()))

nl = [l[0]]
lengl = [1]

```

```

length = 1

for i in range(1, n):
    x = l[i]

    if x > nl[-1]:
        nl.append(x)
        length += 1
        lengl.append(length)
    elif x ≤ nl[0]:
        nl[0] = x
        lengl.append(1)
    else:
        Llimit = 0
        Rlimit = length
        while True:
            index = (Llimit+Rlimit)//2
            if x > nl[index]:
                Llimit = index
            else:
                Rlimit = index
            if Llimit == Rlimit - 1:
                break
        nl[Rlimit] = x
        lengl.append(Rlimit+1)

print(length)

fl = []
for i in range(n-1, -1, -1):
    if lengl[i] == length:
        if len(fl) == 0 or l[i] != fl[-1]:
            length -= 1
            fl.append(l[i])
for i in fl[::-1]:
    print(i, end=' ')

```

1.5 LCS

```

s1 = input()
s2 = input()

```

```

l = [[0]*(len(s2)+1) for i in range(len(s1)+1)]

for i in range(1, len(s1)+1):
    for j in range(1, len(s2)+1):
        if s1[i-1] == s2[j-1]:
            l[i][j] = l[i-1][j-1] + 1
        else:
            l[i][j] = max(l[i-1][j], l[i][j-1])
# end here and print l[-1][-1] if only need length

i = len(s1)
j = len(s2)
lcs = ''
while True:
    if i==0:
        break
    if j==0:
        break
    if l[i-1][j] == l[i][j]:
        i -= 1
        continue
    if l[i][j-1] == l[i][j]:
        j -= 1
        continue
    lcs += s1[i-1]
    i -= 1
    j -= 1
lcs = lcs[::-1]

print(l[-1][-1])
print(lcs)

```

1.6 Disjoint Set

```

class union:
    def __init__(self, size = 10**6):
        self.l = [i for i in range(size)]

    def find(self, i):
        if self.l[i] == i:

```

```

            return i
        self.l[i] = self.find(self.l[i])
        return self.l[i]

    def union(self, i, j):
        self.l[self.find(j)] = self.find(i)
        return

```

1.7 Bellman Ford

```

INF = 1 << 30
v, e = map(int, input().split())
graph = [[] for i in range(v)]
for i in range(e):
    a, b, c = map(int, sys.stdin.readline().split())
    graph[a-1].append((b-1, c))

dist = [INF for i in range(v)]
dist[0] = 0
prv_cities = [0]
for i in range(v):
    cities = []
    for j in prv_cities:
        for k, l in graph[j]:
            x = dist[j] + l
            if dist[k] > x:
                dist[k] = x
                cities.append(k)
    prv_cities = cities

for j in prv_cities:
    for k, l in graph[j]:
        x = dist[j] + l
        if dist[k] > x:
            print(-1)
            sys.exit()

for i in range(1, v):
    print(dist[i] if dist[i] < INF else -1)

```

1.8 ETT & HLD

```

sys.setrecursionlimit(110000)

# Copypaste Segtree Class Here

n = int(input())
edges = [dict() for i in range(n)]
raw_edges = []
for i in range(n-1):
    a, b, w = map(int, input().split()); a -= 1; b -= 1
    edges[a][b] = w
    edges[b][a] = w
    raw_edges.append((a, b))

root = 0
vis = [False] * n; vis[root] = True
parent = [-1] * n
main_child = [-1] * n
depths = [0] * n
def dfs(x):
    tot = 1
    ms = 0; mi = -1
    for j in edges[x]:
        if not vis[j]:
            vis[j] = True
            depths[j] = depths[x] + 1
            parent[j] = x
            rv = dfs(j)
            tot += rv
            if rv > ms:
                ms = rv
                mi = j
    main_child[x] = mi
    return tot
dfs(root)

for i in range(n-1):
    if parent[raw_edges[i][0]] == raw_edges[i][1]:
        raw_edges[i] = raw_edges[i][0]
    else:

```

```

        raw_edges[i] = raw_edges[i][1]

sparse_table = [parent[:]]
for i in range(20):
    ns = []
    for j in range(n):
        if sparse_table[i][j] == -1:
            ns.append(-1)
        else:
            ns.append(sparse_table[i][sparse_table[i][j]])
    sparse_table.append(ns)

def up(x, n):
    for i in range(20):
        if (n >> i) & 1:
            x = sparse_table[i][x]
    return x

def lca(x1, x2):
    dx1 = depths[x1]
    dx2 = depths[x2]
    if dx1 < dx2:
        x2 = up(x2, dx2 - dx1)
    if dx1 > dx2:
        x1 = up(x1, dx1 - dx2)
    for move in range(19, -1, -1):
        if sparse_table[move][x1] != sparse_table[move][x2]:
            x1 = sparse_table[move][x1]
            x2 = sparse_table[move][x2]
    if x1 == x2:
        return x1
    return parent[x1]

idx = 0
hld = []
hldidxl = [-1] * n
hldposl = [-1] * n

vis = [False] * n; vis[root] = True
def dfs2(x):
    global idx

```

```

u = []
e = x

while x != -1:
    vis[x] = True
    hldidxl[x] = idx
    hldposl[x] = len(u)
    u.append(x)
    x = main_child[x]

su = segtree(len(u))
for i in range(len(u)):
    if u[i] == root:
        continue
    su.set(i, edges[parent[u[i]]][u[i]])
su.init()

hld.append((parent[e], su))

for i in u:
    for j in edges[i]:
        if not vis[j]:
            idx += 1
            dfs2(j)

dfs2(root)

for i in range(int(input())):
    q, *ql = map(int, input().split())
    if q == 1:
        i = raw_edges[ql[0]-1]
        hld[hldidxl[i]][1].update(hldposl[i], ql[1])
    if q == 2:
        mv = 0

        c1 = ql[0] - 1
        c2 = ql[1] - 1
        lc = lca(c1, c2)

        while True:
            if hldidxl[c1] == hldidxl[lc]:
                if c1 == lc:

```

```

                    break
                mv = max(mv, hld[hldidxl[c1]][1].query(hldposl[lc]
                    + 1, hldposl[c1]))
                break
            mv = max(mv, hld[hldidxl[c1]][1].query(0,
                hldposl[c1]))
            c1 = hld[hldidxl[c1]][0]

        while True:
            if hldidxl[c2] == hldidxl[lc]:
                if c2 == lc:
                    break
                mv = max(mv, hld[hldidxl[c2]][1].query(hldposl[lc]
                    + 1, hldposl[c2]))
                break
            mv = max(mv, hld[hldidxl[c2]][1].query(0,
                hldposl[c2]))
            c2 = hld[hldidxl[c2]][0]

    print(mv)

```

1.9 SCC

```

INF = 1 << 30
sys.setrecursionlimit(200_000)

n, m = map(int, input().split())
edges = [[] for i in range(n)]
for i in range(m):
    v, u = map(int, input().split()); v -= 1; u -= 1
    edges[v].append(u)

vis = [False] * n
finish = [False] * n
reachable_min = [INF] * n
scc = []
stack = []
def dfs(x):
    global idx
    og_idx = idx
    stack.append(x)

```

```

reachable_min[x] = idx
for i in edges[x]:
    if not vis[i]:
        vis[i] = True
        idx += 1
        reachable_min[x] = min(reachable_min[x], dfs(i))
    else:
        if not finish[i]:
            reachable_min[x] = min(reachable_min[x],
                                   reachable_min[i])
if og_idx == reachable_min[x]:
    t = -1
    vl = []
    while x != t:
        t = stack.pop()
        finish[t] = True
        vl.append(t)
    scc.append(vl)
return reachable_min[x]

for i in range(n):
    if not vis[i]:
        vis[i] = True
        stack = []
        idx = 0
        dfs(i)

```

1.10 2-SAT

```

scc = scc[::-1]

is_okay = True
for i in scc:
    for j in i:
        if j^1 in i:
            is_okay = False
            break
    if not is_okay:
        break
if is_okay:
    print(1)

```

```

else:
    print(0)

if is_okay:
    values = [-1] * n
    for i in scc:
        okay_to_zero = True
        for j in i:
            if values[j] == 1:
                okay_to_zero = False
                break
        if okay_to_zero:
            for j in i:
                values[j] = 0
                values[j^1] = 1
        else:
            for j in i:
                values[j] = 1
                values[j^1] = 0
    for i in values[::2]:
        print(i, end=' ')

```

1.11 Miller Rabin

```

def is_prime(x):
    if x == 2 or x == 3:
        return True
    if x == 1 or x & 1 == 0:
        return False
    if x < 1_373_653:
        prime_test = [2, 3]
    else:
        prime_test = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
    w, r = x - 1, 0
    while w & 1 == 0:
        w >>= 1
        r += 1
    for p in prime_test:
        v = pow(p, w, x)
        if v == 1 or v == x - 1:
            continue

```

```

    for _ in range(r - 1):
        v = pow(v, 2, x)
        if v == 1:
            return False
        if v == x - 1:
            break
    else:
        return False
return True

```

1.12 Pollard Rho

```

def factorize(N):
    if N == 1:
        return []
    if N % 2 == 0:
        l = []
        while N % 2 == 0:
            l.append(2)
            N //= 2
        if N == 1:
            return l
        else:
            return l + factorize(N)
    if is_prime(N):
        return [N]
    while True:
        C = random.randrange(1, N)
        x = random.randrange(1, N)
        l = [x]
        sz = 0
        while True:
            x = (x * x + C) % N
            l.append(x)
            if sz % 2 == 0 and sz > 0:
                g = math.gcd(l[sz] - l[sz//2], N)
                if g != 1:
                    if g == N:
                        break
                    else:
                        return factorize(g) + factorize(N // g)

```

```
sz += 1
```

1.13 Euler Phi

```

def phi(N):
    v = 1
    f = sorted(factorize(N))
    for i in f:
        d = 0
        while N % i == 0:
            d += 1
            N //= i
        if d > 0:
            v *= pow(i, d-1) * (i-1)
    if N > 1:
        v *= N - 1
    return v

```

1.14 Euler Phi (up to N)

```

max_N = 10_000
sieve = list(range(max_N+1))
for i in range(2, max_N+1):
    if i == sieve[i]:
        for j in range(i, max_N+1, i):
            sieve[j] = sieve[j] // i * (i-1)

pfs = []
x = 0
for i in sieve:
    x += i
    pfs.append(x)
for i in range(int(input())):
    i, j = map(int, input().split())
    print(i, pfs[j]+1)

```

1.15 KMP

```

def failure(S):
    sl = len(S)
    f = [0] * sl

```

```

i = 1; j = 0 #i = check (current) index; j = length
while i < sl:
    if S[j] == S[i]:
        f[i] = j + 1
        i += 1
        j += 1
    elif j > 0:
        j = f[j-1]
    else:
        i += 1
return f

def kmp(L, S, F):
    fl = len(F)
    sl = len(S)
    if fl > sl:
        return []
    i = 0; j = 0 # i = current index, j = length found
    ll = []
    while i < sl:
        if F[j] == S[i]:
            i += 1
            j += 1
            if j == fl:
                ll.append(i-j)
                j = L[j-1]
        else:
            if j > 0:
                j = L[j-1]
            else:
                i += 1
    return ll

find = input()
s = input()
l = failure(find)
print(kmp(l, s, find))

```

1.16 Manacher

```
s = input().strip()
```

```

S = '#'+'#'.join(s)+'#'

lg = len(S)

a = []
M = -1
R = -1
mlg = 1
for i in range(lg):
    if i > R:
        a.append(0)
    else:
        a.append(min(a[M*2-i], R - i))
    while True:
        if i-a[i] < 0 or i+a[i] ≥ lg:
            break
        if S[i-a[i]] == S[i+a[i]]:
            a[i] += 1
            continue
        break
    if i + a[i] - 1 > R:
        R = i + a[i] - 1
        M = i
    mlg = max(mlg, a[i]-1)
print(mlg)

```

1.17 CCW & Convex Hull

```
INF = 1 << 63
```

```

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

    def __lt__(self, other):
        return (self.x, self.y) < (other.x, other.y)

    def __add__(self, other):

```

```

        return Point(self.x + other.x, self.y + other.y)

def __sub__(self, other):
    return Point(self.x - other.x, self.y - other.y)

def __mul__(self, other):
    if isinstance(other, Point):
        return self.x * other.x + self.y * other.y
    else:
        return Point(other * self.x, other * self.y)

def __matmul__(self, other):
    return self.x * other.y - self.y * other.x

def __truediv__(self, other):
    if self.x == other.x:
        if self.y == other.y:
            return -INF
        else:
            return INF
    else:
        return (other.y - self.y) / (other.x - self.x)

def __pow__(self, other):
    return abs(self.x) ** other + abs(self.y) ** other

def __abs__(self):
    return pow(self, 2) ** (1 / 2)

def __str__(self):
    return ' '.join((str(self.x), str(self.y)))

def ccw(p1: Point, p2: Point, p3: Point):
    return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x)

def convex_hull(points):
    p0 = min(points)
    points.sort(key=lambda p: (p / p0, (p - p0) ** 1))

```

```

    stack = []
    for point in points:
        while len(stack) > 1 and ccw(stack[-2], stack[-1], point)
            ≤ 0:
            stack.pop()
        stack.append(point)
    return stack

n = int(input())
l = []
for i in range(n):
    l.append(Point(*map(int, input().split())))

ch = convex_hull(l)

1.18 Rotating Calipers

q = convex_hull(l)

lg = len(q)
md = -1
i = 0; j = 1
while i ≤ lg:
    md = max(md, (q[i%lg].x - q[j%lg].x) ** 2 +
        (q[i%lg].y - q[j%lg].y) ** 2)
    c = ccw(q[i%lg], q[(i+1)%lg], q[(i+1)%lg] + q[(j+1)%lg] -
        q[j%lg])
    if c > 0:
        j += 1
    else:
        i += 1
print(math.sqrt(md))

```

1.19 Polygon Overlap Area

```
INF = 1 << 63
```

```

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

```

```

def __eq__(self, other):
    return (self.x, self.y) == (other.x, other.y)

def __lt__(self, other):
    return (self.x, self.y) < (other.x, other.y)

def __add__(self, other):
    return Point(self.x + other.x, self.y + other.y)

def __sub__(self, other):
    return Point(self.x - other.x, self.y - other.y)

def __mul__(self, other):
    if isinstance(other, Point):
        return self.x * other.x + self.y * other.y
    else:
        return Point(other * self.x, other * self.y)

def __matmul__(self, other):
    return self.x * other.y - self.y * other.x

def __truediv__(self, other):
    if self.x == other.x:
        if self.y == other.y:
            return -INF
        else:
            return INF
    else:
        return (other.y - self.y) / (other.x - self.x)

def __pow__(self, other):
    return abs(self.x) ** other + abs(self.y) ** other

def __abs__(self):
    return pow(self, 2) ** (1 / 2)

def __str__(self):
    return ' '.join((str(self.x), str(self.y)))

def __hash__(self):
    return (self.x, self.y).__hash__()

```

```

def ccw(p1: Point, p2: Point, p3: Point):
    return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x)

def ccw_a(p1, p2, p3):
    return sgn(ccw(p1, p2, p3))

def sgn(x):
    if x == 0:
        return x
    else:
        return round(x / abs(x))

def cross(d1, d2, d3, d4):
    x1 = d1.x; y1 = d1.y; x2 = d2.x; y2 = d2.y; x3 = d3.x; y3 = d3.y; x4 = d4.x; y4 = d4.y
    if (ccw_a(d1, d2, d3) == 0 and min(x1, x2) ≤ x3 ≤ max(x1, x2)
        and min(y1, y2) ≤ y3 ≤ max(y1, y2)) or \
        (ccw_a(d1, d2, d4) == 0 and min(x1, x2) ≤ x4 ≤ max(x1, x2)
        and min(y1, y2) ≤ y4 ≤ max(y1, y2)) or \
        (ccw_a(d3, d4, d1) == 0 and min(x3, x4) ≤ x1 ≤ max(x3, x4)
        and min(y3, y4) ≤ y1 ≤ max(y3, y4)) or \
        (ccw_a(d3, d4, d2) == 0 and min(x3, x4) ≤ x2 ≤ max(x3, x4)
        and min(y3, y4) ≤ y2 ≤ max(y3, y4)):
        return True
    elif ccw_a(d1, d2, d3) == ccw_a(d1, d2, d4) or ccw_a(d3, d4, d1) == ccw_a(d3, d4, d2):
        return False
    else:
        return True

def add_dot(p1, p2, p3, p4):
    if cross(p1, p2, p3, p4):
        diff1 = p2 - p1
        diff2 = p4 - p3
        if diff1.x == 0:
            if diff2.x == 0:
                if p1 == p3:
                    if sgn(diff1.y) == sgn(diff2.y):
                        return

```

```

        else:
            dots.append(p1)
        elif p1 == p4:
            if sgn(diff1.y) != sgn(diff2.y):
                return
            else:
                dots.append(p1)
        elif p2 == p3:
            if sgn(diff1.y) != sgn(diff2.y):
                return
            else:
                dots.append(p2)
        elif p2 == p4:
            if sgn(diff1.y) == sgn(diff2.y):
                return
            else:
                dots.append(p2)
        else:
            return
    else:
        x = p1.x
        a = diff2.y / diff2.x
        b = p3.y - a * p3.x
        y = a * x + b
        dots.append(Point(x, y))
    else:
        if diff2.x == 0:
            x = p3.x
            a = diff1.y / diff1.x
            b = p1.y - a * p1.x
            y = a * x + b
            dots.append(Point(x, y))
        else:
            a1 = diff1.y / diff1.x
            b1 = p1.y - a1 * p1.x
            a2 = diff2.y / diff2.x
            b2 = p3.y - a2 * p3.x
            if a1 == a2:
                if p1 == p3:
                    if sgn(diff1.x) == sgn(diff2.x):
                        return

```

```

        else:
            dots.append(p1)
        if p1 == p4:
            if sgn(diff1.x) != sgn(diff2.x):
                return
            else:
                dots.append(p1)
        if p2 == p3:
            if sgn(diff1.x) != sgn(diff2.x):
                return
            else:
                dots.append(p2)
        if p2 == p4:
            if sgn(diff1.x) == sgn(diff2.x):
                return
            else:
                dots.append(p2)
        else:
            return
    else:
        x = (b2-b1) / (a1-a2)
        y = a1 * x + b1
        dots.append(Point(x, y))
    else:
        return

def convex_hull(points):
    p0 = min(points)
    points.sort(key=lambda p: (p / p0, (p - p0) ** 1))
    stack = []
    for point in points:
        while len(stack) > 1 and ccw(stack[-2], stack[-1], point) < 0:
            stack.pop()
        stack.append(point)
    return stack

dots1 = []
dots2 = []
n, m = map(int, input().split())
for i in range(n):

```

```

dots1.append(Point(*map(int, input().split())))
for i in range(m):
    dots2.append(Point(*map(int, input().split())))

dots = []

for i in range(n):
    p1 = dots1[i]; p2 = dots1[(i+1)%n]
    for j in range(m):
        p3 = dots2[j]; p4 = dots2[(j+1)%m]
        add_dot(p1, p2, p3, p4)

for i in dots1:
    flag = True
    for j in range(m):
        if ccw(i, dots2[j], dots2[(j+1)%m]) ≤ 0:
            flag = False
            break
    if flag:
        dots.append(i)

for i in dots2:
    flag = True
    for j in range(n):
        if ccw(i, dots1[j], dots1[(j+1)%n]) ≤ 0:
            flag = False
            break
    if flag:
        dots.append(i)

if len(dots) == 0:
    print(0.0)
    sys.exit()

ch = convex_hull(dots)

lg = len(ch)
area = 0
for i in range(lg):
    area += ch[i] @ ch[(i+1)%lg]
print(abs(area/2))

```

1.20 Ray Casting

```

INF = 1 << 60
EPS = 10 ** (-6)

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        return (self.x, self.y) == (other.x, other.y)

    def __lt__(self, other):
        return (self.x, self.y) < (other.x, other.y)

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Point(self.x - other.x, self.y - other.y)

    def __mul__(self, other):
        if isinstance(other, Point):
            return self.x * other.x + self.y * other.y
        else:
            return Point(other * self.x, other * self.y)

    def __matmul__(self, other):
        return self.x * other.y - self.y * other.x

    def __truediv__(self, other):
        if self.x == other.x:
            if self.y == other.y:
                return -INF
            else:
                return INF
        else:
            return (other.y - self.y) / (other.x - self.x)

    def __pow__(self, other):
        return abs(self.x) ** other + abs(self.y) ** other

```

```

def __abs__(self):
    return pow(self, 2) ** (1 / 2)

def __str__(self):
    return ' '.join((str(self.x), str(self.y)))

def __hash__(self):
    return (self.x, self.y).__hash__()

origin = Point(0, 0)

def ccw(p1: Point, p2: Point, p3: Point):
    return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x)

def ccw_a(p1, p2, p3):
    return sgn(ccw(p1, p2, p3))

def sgn(v):
    if v > 0:
        return 1
    if v < 0:
        return -1
    return 0

def cross(d1, d2, d3, d4):
    x1, y1 = d1.x, d1.y
    x2, y2 = d2.x, d2.y
    x3, y3 = d3.x, d3.y
    x4, y4 = d4.x, d4.y
    if (-EPS ≤ ccw_a(d1, d2, d3) ≤ EPS and min(x1,x2) - EPS ≤
        x3 ≤ max(x1,x2) + EPS and min(y1,y2) - EPS ≤ y3 ≤
        max(y1,y2) + EPS) or \
        (-EPS ≤ ccw_a(d1, d2, d4) ≤ EPS and min(x1,x2) - EPS ≤
        x4 ≤ max(x1,x2) + EPS and min(y1,y2) - EPS ≤ y4 ≤
        max(y1,y2) + EPS) or \
        (-EPS ≤ ccw_a(d3, d4, d1) ≤ EPS and min(x3,x4) - EPS ≤
        x1 ≤ max(x3,x4) + EPS and min(y3,y4) - EPS ≤ y1 ≤
        max(y3,y4) + EPS) or \

```

```

        (-EPS ≤ ccw_a(d3, d4, d2) ≤ EPS and min(x3,x4) - EPS ≤
        x2 ≤ max(x3,x4) + EPS and min(y3,y4) - EPS ≤ y2 ≤
        max(y3,y4) + EPS):
            return True
    elif -EPS ≤ ccw_a(d1, d2, d3) - ccw_a(d1, d2, d4) ≤ EPS or
        -EPS ≤ ccw_a(d3, d4, d1) - ccw_a(d3, d4, d2) ≤ EPS:
        return False
    else:
        return True

def line_cross_dist(p1, p2, p3, p4):
    if cross(p1, p2, p3, p4):
        diff1 = p2-p1
        diff2 = p4-p3
        if diff2.x == 0:
            x = p3.x
            a = diff1.y / diff1.x
            b = p1.y - a * p1.x
            y = a * x + b
            pt = Point(x, y)
            return abs(p1-pt), pt
        else:
            a1 = diff1.y / diff1.x
            b1 = p1.y - a1 * p1.x
            a2 = diff2.y / diff2.x
            b2 = p3.y - a2 * p3.x
            x = (b2-b1) / (a1-a2)
            y = a1 * x + b1
            pt = Point(x, y)
            return abs(p1-pt), pt
    return INF, Point(-1, -1)

def sort_order_1(e):
    Pt1 = e[0] - light
    typ = e[2]
    if -EPS < Pt1.x < EPS: Pt1.x = 0
    if -EPS < Pt1.y < EPS: Pt1.y = 0
    v1 = [6,5,4,7,0,3,8,1,2][sgn(Pt1.x)*3+sgn(Pt1.y)+4]
    if typ == 2:
        ue = -abs(Pt1)
    else:

```

```

        ue = abs(Pt1)
    if typ == 0:
        typ = 1.5
    return (v1, round(float(Pt1/origin), 7), typ, ue)

def polygon_area(pts):
    a = 0
    lg = len(pts)
    for i in range(lg):
        a += pts[i] @ pts[(i+1)%lg]
    return abs(a/2)

n, m = map(int, input().split())
l = [input().strip() for i in range(n)]

id_ = 0
box_lines = []
box_dots = []
light_i = -1
for i in range(n):
    for j in range(m):
        if l[i][j] == '*':
            light = Point(j+0.5, i+0.5)
            light_i = i
        if l[i][j] == '#':
            box_lines_t = []
            box_lines_t.append((Point(j, i), Point(j, i+1)))
            box_lines_t.append((Point(j, i+1), Point(j+1, i+1)))
            box_lines_t.append((Point(j+1, i+1), Point(j+1, i)))
            box_lines_t.append((Point(j+1, i), Point(j, i)))
            box_lines.append(box_lines_t)
            if i == light_i:
                box_dots.append((Point(j, i), id_, 1))
                box_dots.append((Point(j, i+1), id_, 1))
                box_dots.append((Point(j+1, i+1), id_, 1))
                box_dots.append((Point(j+1, i), id_, 1))
            else:
                box_dots.append((Point(j, i), id_, 0))
                box_dots.append((Point(j, i+1), id_, 0))
                box_dots.append((Point(j+1, i+1), id_, 0))
                box_dots.append((Point(j+1, i), id_, 0))

```

```

        id_ += 1
    box_dots = sorted(box_dots, key = sort_order_1)

    found_dots = [0] * id_
    sweep_dots = [] # 0: 자기만 확인, 1: inner->outer, 2: outer->inner
    box_angles = [set() for i in range(id_ + 1)]
    for i, j, k in box_dots:
        found_dots[j] += 1
        d = abs(light-i)
        md = d
        for l in box_lines[j]:
            md = min(md, line_cross_dist(light, i, l[0], l[1])[0])
        if md + EPS >= d:
            if k == 0 and found_dots[j] == 1:
                sweep_dots.append((i, j, 2))
                box_angles[j].add(round(light/i, 7))
            elif k == 0 and found_dots[j] == 4:
                sweep_dots.append((i, j, 1))
                box_angles[j].add(round(light/i, 7))
            elif k == 1 and found_dots[j] == 2:
                sweep_dots.append((i, j, 1))
                box_angles[j].add(round(light/i, 7))
            elif k == 1 and found_dots[j] == 3:
                sweep_dots.append((i, j, 2))
                box_angles[j].add(round(light/i, 7))
            else:
                sweep_dots.append((i, j, 0))
    sweep_dots.append((Point(0, 0), id_, 0))
    sweep_dots.append((Point(m, 0), id_, 0))
    sweep_dots.append((Point(m, n), id_, 0))
    sweep_dots.append((Point(0, n), id_, 0))
    sweep_dots = sorted(sweep_dots, key = sort_order_1)

    frame = []
    frame.append((Point(0, 0), Point(m, 0)))
    frame.append((Point(m, 0), Point(m, n)))
    frame.append((Point(m, n), Point(0, n)))
    frame.append((Point(0, n), Point(0, 0)))
    box_lines.append(frame)
    line_amt = len(box_lines)

```

```

final_dots = []
for pt, box_id, typ in sweep_dots:
    angle = round(light/pt, 7)
    if typ == 0:
        d = abs(light-pt)
        flag = False
        for j in range(line_amt):
            for k1, k2 in box_lines[j]:
                if line_cross_dist(light, pt, k1, k2)[0] + EPS <
                    d:
                    flag = True
                    break
            if flag:
                break
        if not flag:
            final_dots.append(pt)
    elif typ == 1:
        d = abs(light-pt)
        pt_extend = light + (pt-light) * 1000
        md = INF
        md_pt = Point(-1, -1)
        for j in range(line_amt):
            if angle in box_angles[j]:
                continue
            for k1, k2 in box_lines[j]:
                dd, dd_pt = line_cross_dist(light, pt_extend, k1,
                    k2)
                if dd < md:
                    md = dd
                    md_pt = dd_pt
            if md ≥ d - EPS:
                final_dots.append(pt)
                final_dots.append(md_pt)
    elif typ == 2:
        d = abs(light-pt)
        pt_extend = light + (pt-light) * 1000
        md = INF
        md_pt = Point(-1, -1)
        for j in range(line_amt):
            if angle in box_angles[j]:
                continue

```

```

        for k1, k2 in box_lines[j]:
            dd, dd_pt = line_cross_dist(light, pt_extend, k1,
                k2)
            if dd < md:
                md = dd
                md_pt = dd_pt
        if md ≥ d - EPS:
            final_dots.append(md_pt)
            final_dots.append(pt)

```

```

final_area = polygon_area(final_dots)
print(n*m - id_ - final_area)

```

1.21 Min Enclosing Circle

```

def get_outer_circle(pt1, pt2, pt3):
    x1, y1, x2, y2, x3, y3 = pt1.x, pt1.y, pt2.x, pt2.y, pt3.x,
        pt3.y
    x_parent = 2*(x1-x2)*(y3-y2)-2*(x3-x2)*(y1-y2)
    y_parent = -x_parent
    x_child =
        (x2*x2-x1*x1+y2*y2-y1*y1)*(y3-y2)-(x2*x2-x3*x3+y2*y2-y3*y3)*(y1-y2)
    y_child =
        (y2*y2-y1*y1+x2*x2-x1*x1)*(x3-x2)-(y2*y2-y3*y3+x2*x2-x3*x3)*(x1-x2)
    outer_center = Point(-x_child/x_parent, -y_child/y_parent)
    dist = abs(pt1-outer_center)
    return outer_center, dist

```

```

n = int(input())
points = []
for i in range(n):
    points.append(Point(*map(float, input().split())))
random.shuffle(points)

```

```

if n == 1:
    print(points[0].x, points[0].y, 0)
    sys.exit()

```

```

c, r = Point(0, 0), 0
for i in range(n):
    if abs(points[i]-c) ≤ r:

```

```

        continue
    c, r = points[i], 0
    for j in range(i):
        if abs(points[j]-c) ≤ r:
            continue
        c, r = (points[i]+points[j])*0.5,
        abs(points[i]-points[j])*0.5
    for k in range(j):
        if abs(points[k]-c) ≤ r:
            continue
        c, r = get_outer_circle(points[i], points[j],
        points[k])
    print(c.x, c.y, r)

```

1.22 Dinic

```

from collections import deque
INF = 1 << 63

```

```

N, P = map(int, input().split())

```

```

V = N # system size
connect = [[] for _ in range(V)]
flow = [{ } for _ in range(V)]
capacity = [{ } for _ in range(V)]

```

```

def add_edge(u, v, cap):
    connect[u].append(v)
    flow[u][v] = 0
    capacity[u][v] = cap

```

```

for _ in range(P):
    u, v = map(int, input().split())
    add_edge(u - 1, v - 1, 1)
    add_edge(v - 1, u - 1, 0)

```

```

source, sink = 0, 1
net_flow = 0

```

```

def bfs():
    level[:] = [-1] * V
    level[source] = 0
    queue = deque([source])

    while queue:
        u = queue.popleft()
        for v in connect[u]:
            if capacity[u][v] - flow[u][v] > 0 & level[v]:
                queue.append(v)
                level[v] = level[u] + 1
    return level[sink] ≥ 0

```

```

def dfs(u, max_flow):
    if u == sink:
        return max_flow
    try:
        while True:
            v = next(it[u])
            if level[v] == level[u] + 1 and capacity[u][v] >
            flow[u][v]:
                flow_change = dfs(v, min(capacity[u][v] - flow[u]
                [v], max_flow))
                if flow_change > 0:
                    flow[u][v] += flow_change
                    flow[v][u] -= flow_change
                    return flow_change
    except StopIteration:
        return 0

```

```

level = [-1] * V
while bfs():
    it = [*map(iter, connect)]
    while True:
        flow_change = dfs(source, INF)
        if flow_change == 0:
            break
        net_flow += flow_change

```

```
print(net_flow)
```

1.23 Tutte Matrix

```
P = 2_147_483_647
```

```
def add_mat(a, b):
    for i in range(N):
        mat[a][i] = (mat[a][i] + mat[b][i]) % P
def mul_mat(a, b):
    for i in range(N):
        mat[a][i] = (mat[a][i] * b) % P
def sub_mat(a, b):
    for i in range(N):
        mat[a][i] = (mat[a][i] - mat[b][i]) % P
def count_nonzero_mat():
    cnt = 0
    for i in range(N):
        if sum(mat[i]) != 0:
            cnt += 1
    return cnt

N, M = map(int, input().split())
mat = [[0] * N for i in range(N)]

for i in range(M):
    a, b = sorted(list(map(int, input().split()))); a -= 1; b -= 1
    x = random.randrange(1, P)
    mat[a][b] = x; mat[b][a] = (-x)%P

for x in range(N):
    for y in range(N-1):
        if mat[y][x] == 0:
            continue
        if mat[y+1][x] == 0:
            add_mat(y+1, y)
        v1 = mat[y][x]; v2 = mat[y+1][x]
        mul_mat(y, v2); mul_mat(y+1, v1)
        sub_mat(y, y+1)
```

```
if count_nonzero_mat() == N:
    print(1)
else:
    print(0)
```

2 Data Structure

2.1 PBDS

Time Complexity: $\mathcal{O}(\log N)$

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

//중복 비허용 int형 pbds
#define ordered_set tree<int, null_type, less<>, rb_tree_tag,
tree_order_statistics_node_update>
//중복 허용 int형 pbds
#define ordered_set tree<int, null_type, less_equal<>,
rb_tree_tag, tree_order_statistics_node_update>

ordered_set OS; //pbds OS 선언
OS.insert(val); // OS에 val 삽입
OS.erase(val); // OS에서 val 삭제 (중복 비허용일때)
OS.order_of_key(val); // OS에서 val보다 작은 원소의 개수 리턴
OS.find_by_order(K); // OS[K] 리턴 (0-based-index)

void erase(ordered_set &OS, int val){ // OS에서 val 삭제 (중복 허용일
때)
    auto it = OS.find_by_order(OS.order_of_key(val));
    if (*it == val) OS.erase(it);
}
```

3 Graph & Flow

3.1 이분 매칭(Kuh, 느림)

Time Complexity: $\mathcal{O}(N \log N)$

```

vector<int> graph[total_graph_sz];
int from[right_graph_sz], visited[left_graph_sz];
int visitco; //매번 dfs 돌때마다 값 1씩 증가

bool dfs(int node){
    if (visited[node] == visitco) return false;
    visited[node] = visitco;
    for (int i : graph[node]){
        if (from[i] == -1){
            from[i] = node;
            return true;
        }
        if (dfs(from[i])){
            from[i] = node;
            return true;
        }
    }
    return false;
}

```

4 Fast Fourier Transform

4.1 Fast Fourier Transform

Time Complexity: $\mathcal{O}(N \log N)$

```

typedef complex<long double> cld;

// 곱할 두 벡터를 isfft true로 하고 각각 fft 돌린뒤에 자릿수별로 곱해주고
isfft false로 하고 fft 돌리기
vector<cld> fft(vector<cld>& a, bool isfft){
    int n = 1, isf = isfft ? 2 : -2;
    while (n < a.size()) n <<= 1;
    vector<cld> ret(n);

    for (int i = 0; i < n; i++){
        int tmp = 0, j = 1, rj = n >> 1;
        for (; j < n; j <<= 1, rj >>= 1){
            if (i & j) tmp |= rj;
        }
        if (i < a.size()) ret[tmp] = a[i];
    }
}

```

```

}

for (int i = 2; i <= n; i <<= 1){
    cld w = exp(cld(0, isf * M_PI / i));
    int ord = i >> 1;
    for (int j = 0; j < n; j += i){
        cld wn = cld(1, 0);
        for (int k = j; k < (j | ord); k++, wn *= w){
            cld tmp = ret[k | ord];
            ret[k | ord] = ret[k] - wn * tmp;
            ret[k] += wn * tmp;
        }
    }
}

if (!isfft) for (cld& i : ret) i /= n;

return ret;
}

```

5 ETC

5.1 Template

```

#pragma GCC optimize("Ofast")
#pragma GCC optimize("unroll-loops")
#include <bits/stdc++.h>
#include <cassert>
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

}

```

5.2 Useful Stuff

- Catalan Number
 $1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900$
 $C_n = \text{binomial}(n * 2, n) / (n + 1);$
 - 길이가 $2n$ 인 올바른 괄호 수식의 수
 - $n + 1$ 개의 리프를 가진 풀 바이너리 트리의 수
 - $n + 2$ 각형을 n 개의 삼각형으로 나누는 방법의 수
- Burnside's Lemma
 경우의 수를 세는데, 특정 transform operation(회전, 반사, ..) 해서 같은 경우들은 하나로 친다. 전체 경우의 수는? 각 operation마다 이 operation을 했을 때 변하지 않는 경우의 수를 센다 (단, "아무것도 하지 않는다" 라는 operation도 있어야 함!) 전체 경우의 수를 더한 후, operation의 수로 나눈다. (답이 맞다면 항상 나누어 떨어져야 한다)
- 알고리즘 게임
 - Nim Game의 해법 : 각 더미의 돌의 개수를 모두 XOR했을 때 0 이 아니면 첫번째, 0 이면 두번째 플레이어가 승리.
 - Grundy Number : 어떤 상황의 Grundy Number는, 가능한 다음 상황들의 Grundy Number를 모두 모은 다음, 그 집합에 포함 되지 않는 가장 작은 수가 현재 state의 Grundy Number가 된다. 만약 다음 state가 독립된 여러개의 state들로 나뉘는 경우, 각각의 state의 Grundy Number의 XOR 합을 생각한다.
 - Subtraction Game : 한 번에 k 개까지의 돌만 가져갈 수 있는 경우, 각 더미의 돌의 개수를 $k + 1$ 로 나눈 나머지를 XOR 합하여 판단한다.
 - Index-k Nim : 한 번에 최대 k 개의 더미를 골라 각각의 더미에서 아무렇게나 돌을 제거할 수 있을 때, 각 binary digit에 대하여 합을 $k + 1$ 로 나눈 나머지를 계산한다. 만약 이 나머지가 모든 digit에 대하여 0이라면 두번째, 하나라도 0이 아니라면 첫번째 플레이어가 승리.
- Pick's Theorem
 격자점으로 구성된 simple polygon이 주어짐. I 는 polygon 내부의 격자점 수, B 는 polygon 선분 위 격자점 수, A 는 polygon의 넓이라고 할 때, 다음과 같은 식이 성립한다. $A = I + B/2 - 1$
- 가장 가까운 두 점 : 분할정복으로 가까운 6개의 점만 확인
- 홀의 결혼 정리 : 이분그래프(L-R)에서, 모든 L을 매칭하는 필요충분 조건 = L에서 임의의 부분집합 S를 골랐을 때, 반드시 (S의 크기) $\leq$ (S와 연결되어있는 모든 R의 크기)이다.
- 소수 : 10 007 , 10 009 , 10 111 , 31 567 , 70 001 , 1 000 003 , 1 000 033 , 4 000 037 , 99 999 989 , 999 999 937 , 1 000 000 007 , 1 000 000 009 , 9 999 999 967

, 99 999 999 977

- 소수 개수 : (1e5 이하 : 9592), (1e7 이하 : 664 579) , (1e9 이하 : 50 847 534)
- 10^{15} 이하의 정수 범위의 나눗셈 한번은 오차가 없다.
- N 의 약수의 개수 = $O(N^{1/3})$, N 의 약수의 합 = $O(N \log \log N)$
- $\phi(mn) = \phi(m)\phi(n)$, $\phi(pr^n) = pr^n - pr^{n-1}$, $a^{\phi(n)} \equiv 1 \pmod{n}$ if coprime
- Euler characteristic : $v - e + f$ (면, 외부 포함) = $1 + c$ (컴포넌트)
- Euler's phi $\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$
- Lucas' Theorem $\binom{m}{n} \equiv \prod \binom{m_i}{n_i} \pmod{p}$ m_i, n_i 는 p^i 의 계수
- 스케줄링에서 데드라인이 빠른 걸 쓰는 게 이득. 늦은 스케줄이 안들어갈 때 가장 시간 소모가 큰 스케줄 1개를 제거하면 이득.

5.3 자주 쓰이는 문제 접근법

- 비슷한 문제를 풀어본 적이 있던가?
- 단순한 방법에서 시작할 수 있을까? (brute force)
- 내가 문제를 푸는 과정을 수식화할 수 있을까? (예제를 직접 해결해보면서)
- 문제를 단순화할 수 있을까?
- 그림으로 그려볼 수 있을까?
- 수식으로 표현할 수 있을까?
- 문제를 분해할 수 있을까?
- 뒤에서부터 생각해서 문제를 풀 수 있을까?
- 순서를 강제할 수 있을까?
- 특정 형태의 답만을 고려할 수 있을까? (정규화)
- 특수 조건을 꼭 활용
- 여사건으로 생각하기
- 게임이론 - 거울 전략 혹은 mex DP 연계

- 겁먹지 말고 경우 나누어 생각
- 해법에서 역순으로 가능한가?
- 딱 맞는 시간복잡도에 집착하지 말자
- 문제에 의미있는 작은 상수 이용
- 스몰투라지, 트라이, 해싱, 루트질 같은 트릭 생각
- 너무 추상화하기보단 풀려야 하는 방식으로 생각하기
- 잘못된 방법으로 파고들지 말고 버리자
- 제발 터널 비전에 빠지지 말자
- 헬프 콜은 적극적으로
- 혼자 멘탈 나가지 않기

5.4 DP 최적화 접근

- $C[i, j] = A[i] * B[j]$ 이고 A, B가 단조증가, 단조감소이면 Monge
- l..r의 값들의 sum이나 min은 Monge
- 식 정리해서 일차(CHT) 혹은 비슷한(MQ) 함수를 발견, 구현 힘들면 Li-Chao
- $a \leq b \leq c \leq d$ 에서 $A[a, c] + A[b, d] \leq A[a, d] + A[b, c]$
- Monge 성질을 보이기 어려우면 N^2 나이트 짜서 opt의 단조성을 확인하고 짚맞
- 식이 간단하거나 변수가 독립적이면 DP 테이블을 세그 위에 올려서 해결
- 침착하게 점화식부터 세우고 Monge인지 판별
- Monge에 집착하지 말고 단조성이나 볼록성만 보여도 됨